VALOR

Write with clarity. Validate before translation.

VALOR is a statically typed programming language with a source-to-source transpiler that translates VALOR programs into readable JavaScript. The generated JavaScript can then be executed using Node.js.

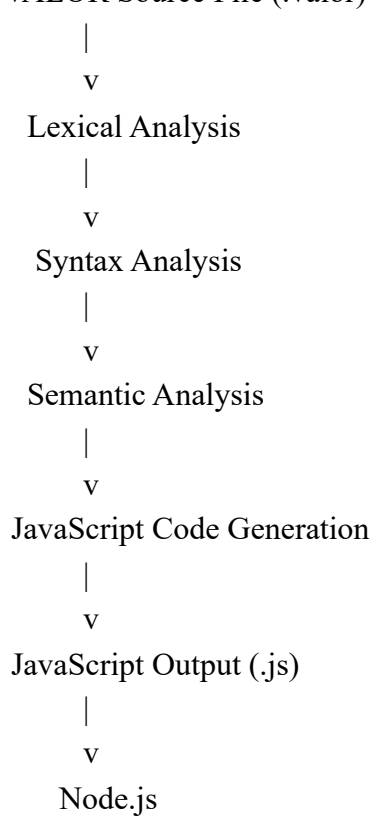
Overview

VALOR is designed to provide a simple, readable programming syntax while enforcing type rules before code generation.

The transpilation process follows four main stages:

1. **Lexical Analysis** -- converts source code into tokens.
2. **Syntax Analysis** -- checks the token sequence against VALOR's grammar and builds an Abstract Syntax Tree (AST).
3. **Semantic Analysis** -- checks declarations, types, and other language rules.
4. **JavaScript Code Generation** -- converts the validated AST into JavaScript.

A VALOR program is translated only after it passes the required compilation stages.



Language Features

The language design includes the following features. Actual support depends on the current implementation and should be verified with the examples and tests in this repository.

- Explicit variable type annotations
- Basic types: number, string, and boolean
- Variable declarations using let
- Arithmetic and comparison expressions
- Conditional statements using if and else
- Repetition using while
- Function declarations and return statements
- Output statements using reveal(...)
- Single-line comments beginning with //

Example

```
let studentName: string = "Reimar";
let score: number = 92;
let passingScore: number = 75;
let passed: boolean = score >= passingScore;

reveal("=== VALOR GRADE EVALUATOR ===");
reveal(studentName);
reveal(score);

if (passed) {
   reveal("Result: PASSED");
} else {
   reveal("Result: FAILED");
}
```

The reveal(...) statement is VALOR's output instruction. It is translated into JavaScript's console.log(...).

Type Rules

VALOR uses explicit type annotations. A variable declaration specifies its type:

```
let age: number = 21;
let name: string = "Alex";
let isEnrolled: boolean = true;
```

The initializer must be compatible with the declared type. VALOR does not rely on automatic type inference in this design.

The + operator supports numeric addition (number + number) and string concatenation (string + string). Mixing a number and a string with + is rejected by semantic analysis.

Requirements

Install the following:

- **Node.js** -- required to run the VALOR transpiler and execute generated JavaScript.
- **Visual Studio Code** or another code editor -- recommended for editing .valor source files.

Check that Node.js and npm are available:

node --version
npm --version

Project Structure

The project is organized into source modules, example programs, generated output, and documentation.

```
valor/
├── src/
│   ├── main.js
│   ├── lexer.js
│   ├── parser.js
│   ├── semantics.js
│   ├── codegen.js
│   └── test_lexer.js
├── examples/
│   ├── hello.valor
│   └── signature.valor
├── dist/
│   └── (generated JavaScript files)
├── docs/
├── package.json
└── README.md
```

Your local project may contain additional test files or generated files.

Getting Started

1. Open the project

Open the valor project folder in Visual Studio Code.

Open the integrated terminal using **Terminal → New Terminal**. Make sure the terminal is in the project root---the directory containing src, examples, and package.json.

If necessary, navigate to the project folder:

cd path/to/valor

2. Run the transpiler

Run:

node src/main.js

The main program reads the source file configured in src/main.js, runs it through the transpilation stages, and writes the generated JavaScript file to dist/.

Important: The input and output filenames are currently determined by the paths configured in src/main.js. For example, if the file is configured to read examples/hello.valor, it will generate dist/hello.js. If it is configured to read examples/signature.valor, it should write to the corresponding output filename.

3. Execute the generated JavaScript

Run the generated file using Node.js. For example:

node dist/hello.js

Or, if your main.js generates signature.js:

node dist/signature.js

Use the filename that was actually generated in your dist/ directory.

Creating and Running Another VALOR Program

1. Create a new file with the .valor extension inside examples/.
2. Write a program using VALOR syntax.
3. Update the source and output paths in src/main.js to point to your new file.
4. Run the transpiler:
5. Execute the generated JavaScript file with Node.js.

For example, to compile examples/my_program.valor, configure src/main.js to read that file and write an appropriately named .js file into dist/.

Error Handling

The transpiler reports compilation errors when a stage cannot process the program. Examples include:

- **Lexical errors:** an unrecognized character or invalid token.
- **Syntax errors:** a token sequence that does not follow the language grammar.
- **Semantic errors:** a type mismatch or another violation of a language rule.

A semantic error such as using number + string with the + operator should be corrected in the VALOR source. The transpiler should reject the invalid program rather than generate JavaScript from it.

Testing

The repository includes a lexer test file named src/test_lexer.js. If it is a standalone Node.js test script, run it from the project root with:

node src/test_lexer.js

Other tests should be run using the commands and test files actually present in the project. Do not assume a test suite exists until it has been added and verified.

Implementation Notes

This README describes the intended language design and the project structure previously established for VALOR. Some language features may still be under development. Confirm each feature by running a corresponding .valor example and checking the generated JavaScript and runtime behavior.

In particular, verify function return-path checking, recursion, loops, assignment behavior, and division-by-zero handling before presenting them as fully implemented capabilities.

Author

Reimar Escueta

Master of Science in Computer Science
Technological Institute of the Philippines -- Manila